

**IFQ716: Advanced Web Development**  
**Assignment 2: Node.js and Express-based API**  
Matthew Campbell  
20.06.24  
Queensland University of Technology

## Introduction

My work for Assignment 2 has culminated in something resembling a functioning full-stack API. All endpoints have been implemented in line with the Swagger Movie Information API documentation<sup>1</sup> to varying degrees of robustness using Express. Knex was utilised for displaying information at the movie search and data endpoints as well as for saving user information. Where information in the database was either lacking or not unuseful, the API contains a fetch call to the OMDb API<sup>2</sup>, for example in fleshing out the information at the movies/data endpoint.

In addition to attempting to satisfy all the requirements for this assignment, I have also tried to implement a functioning front-end using the Handlebars templating engine<sup>3</sup>. I was motivated to do this to improve my front-end skills, but also to provide an alternative, more user-friendly way of testing the app. The app is nevertheless fully testable in Insomnia. The app has not been deployed to a URL for reasons mentioned later in this report.

## Technical description of the application

My app was architected in line with the Express application completed as part of this course<sup>4</sup>. Other architectural considerations included having a 'utils' folder for storing functions for fetching information from the database and the API which was influenced by Andrew Mead's *The Complete Node.js Developer Course (3rd Edition)* on Udemy<sup>5</sup>.

The functions for fetching data from OMDb were created along similar lines to my work in Assignment 1. Building the queries for getting information from the database did prove a challenge for me given my still limited knowledge of SQL. The functions were written asynchronously, taking the database and searchTerm objects as

---

<sup>1</sup> <http://54.79.30.138:3001/#/>

<sup>2</sup> <https://www.omdbapi.com/>

<sup>3</sup> <https://handlebarsjs.com/>

<sup>4</sup> [https://canvas.qutonline.edu.au/courses/1604/pages/5-dot-3-activity-1-installing-express?module\\_item\\_id=92543](https://canvas.qutonline.edu.au/courses/1604/pages/5-dot-3-activity-1-installing-express?module_item_id=92543)

<sup>5</sup> <https://www.udemy.com/course/the-complete-nodejs-developer-course-2/?couponCode=24T6MT62024>

parameters. The functions return the database, then, using the `.select` method specify the required information, and finally in the `.where` method match<sup>6</sup> the required field with `searchTerm` using the `like` operator<sup>7</sup>.

Keeping the database and OMDb fetch functions separate helped keep my code in general modular and clean; likewise for the middleware and configuration files in their respective folders. One area where I wonder if my code could have been cleaner, however, was in the router functions. Without knowing much about front-end programming, Handlebars provided me with a fairly straightforward way of developing a working front-end, especially with its built-in helpers like the `each`<sup>8</sup> and `if`-statements<sup>9</sup>. However, rendering the content did start to make my code look a bit cumbersome.

If that wasn't cumbersome enough, attempts to merge the data for the movies/data endpoint in a similar way to how I implemented this in Assignment 1 didn't yield good results, and ultimately I had to construct a rather awkward-looking `if`-statement with a nested `for`-loop which itself contains its own `if`-statement (see `movies.js`).

My solutions for getting and uploading posters, although incomplete, are an improvement on the work I did for Assignment 1. Fetching the posters was a simple matter of constructing an asynchronous fetch call to the OMDb API. I even managed to get them rendering dynamically<sup>10</sup> in the browser. Uploading the posters was naturally more of a challenge, but provided me with some good experience in configuring middleware and understanding its usefulness, in this case `multer`<sup>11</sup>. The middleware is configured in the `multerUpload.js` file and imported into `posters.js`. I initially had some difficulty figuring out the best way to construct the filename object<sup>12</sup>, but ultimately I believe my solution does the job of making the file names sufficiently unique so that in theory there would be no issues with duplicates.

One problem I wanted to solve regarding the poster uploads was how to get the images onto the database. In researching this problem, I came across the idea of storing them as BLOBs (Binary Large Objects), but was put off by the idea that storing images directly in the database could impact its performance<sup>13</sup>. To avoid this, I experimented with storing the file path instead, but couldn't find much in my research<sup>14</sup> to help me, and consulting the `knex` documentation only yielded potentially (but in my case

---

<sup>6</sup> [https://www.w3schools.com/sql/sql\\_like.asp](https://www.w3schools.com/sql/sql_like.asp)

<sup>7</sup> <https://knexjs.org/guide/query-builder.html#where>

<sup>8</sup> <https://handlebarsjs.com/guide/builtin-helpers.html#unless>

<sup>9</sup> <https://www.sitepoint.com/a-beginners-guide-to-handlebars/>

<sup>10</sup> <https://stackoverflow.com/questions/12531743/handlebars-templating-and-dynamic-images>

<sup>11</sup> <https://www.npmjs.com/package/multer>

<sup>12</sup> <https://medium.com/@livajorge7/a-comprehensive-guide-to-uploading-and-saving-images-with-node-js-express-js-and-javascript-69137e227c4d>

<sup>13</sup> <https://code.likeagirl.io/can-sql-store-images-2266304dcddb>

<sup>14</sup> <https://dev.to/cesareferrari/creating-data-with-knex-in-a-rest-api-295e>

ultimately not) useful methods<sup>15</sup>. Inoperative database upload code notwithstanding, images still upload to local storage with the help of multer.

Registration and upload routes were written in line with the course material<sup>16</sup>, but with more robust error handling, modularisation<sup>17</sup> and separation of concerns<sup>18</sup>. Middleware functions for generating and authenticating access tokens are found in auth.js.

## Testing and limitations

Testing was done in-browser, using Insomnia and to a lesser extent by observing logs created with the winston logger<sup>19</sup>.

In-browser testing using DevTools was particularly useful in spotting errors related to CSP issues, which arose at various junctures and became particularly pertinent after installing helmet to its default configurations<sup>20</sup>. Since helmet blocks images from outside sources, I had to manually configure its 'directives' object<sup>21</sup> to accept images coming from the URL at the posters endpoint.

In a similar way I was able to ascertain why my dark mode function wasn't working, leading me to stumble across the concept of hashing and nonces for allowing scripts<sup>22</sup>. As the dark mode function was not necessary for completing the assignment, I have decided to return to this problem at another time.

I've tried my best to ensure that my app is capable of handling a range of typical errors through religious use of try/catch blocks and fit-for-purpose conditional logic across all endpoints. For the sake of brevity I comment here on some of the error handling, with particular focus on the more sophisticated examples. A demonstration of the full range of errors and edge-case handling can be found in my video submission.

### No search term and search term 'undefined' (movies.js, posters.js)

My initial if-statement was, frustratingly, returning the error message by default (that is, even when just visiting the page, before even searching for anything):

---

<sup>15</sup> <https://knexjs.org/guide/query-builder.html#update>

<sup>16</sup> [https://canvas.qutonline.edu.au/courses/1604/pages/7-dot-6-activity-2-registration-and-login?module\\_item\\_id=92562](https://canvas.qutonline.edu.au/courses/1604/pages/7-dot-6-activity-2-registration-and-login?module_item_id=92562)

<sup>17</sup> <https://www.digitalocean.com/community/tutorials/nodejs-jwt-expressjs>

<sup>18</sup>

[https://help.sap.com/doc/abapdocu\\_753\\_index\\_htm/7.53/en-US/abenseperation\\_concerns\\_guidl.htm#:~:text=Separation%20of%20concerns%20is%20a, and%20arrangement%20in%20software%20layers.](https://help.sap.com/doc/abapdocu_753_index_htm/7.53/en-US/abenseperation_concerns_guidl.htm#:~:text=Separation%20of%20concerns%20is%20a, and%20arrangement%20in%20software%20layers.)

<sup>19</sup> <https://betterstack.com/community/guides/logging/how-to-install-setup-and-use-winston-and-morgan-to-log-node-js-applications/>

<sup>20</sup> <https://www.npmjs.com/package/helmet>

<sup>21</sup> <https://blog.logrocket.com/using-helmet-node-js-secure-application/>

<sup>22</sup> <https://lebcit.github.io/posts/csp-nonce-with-nodejs-and-ejs/>

```

if (!searchTerm || searchTerm.trim() === "") {
  return res.status(400).render("movies/search", {
    noTitleError: "You must provide a title!",
    notFoundError: "",
    error: "",
    combinedInfo: [],
    searchTerm: "",
  });
}

```

This I managed to solve by including another if-statement to check if searchTerm is exactly 'undefined' and rendering nothing in the return statement:

```

if (searchTerm === undefined) {
  return res.render("movies/search", {
    noTitleError: "",
    notFoundError: "",
    error: "",
    combinedInfo: [],
    searchTerm: "",
  });
}
};

```

One peculiarity that arose was whitespace, which was handled using the .trim method<sup>23</sup>.

### No movies found (movies.js, posters.js)

The movies/search and movies/data endpoints had to account both for the information coming from the database and from the API. It was important therefore to include checks for both:

```

const dbInfo = await getDBimdbIDInfo(req.db, searchTerm);
const omdbInfo = await getOMDBimdbIDInfo(searchTerm);

if (dbInfo.length === 0 || (!omdbInfo || omdbInfo.Response === "False")) {
  return res.status(404).render("movies/data", {
    notFoundError: "Movie not found. Please try a different title.",
    noIdError: "",
    error: "",
    combinedInfo: [],
    searchTerm,
  });
}
};

```

<sup>23</sup> <https://johnkavanagh.co.uk/articles/check-if-a-string-contains-only-whitespace-with-javascript/>

Since the select method returns an array<sup>24</sup>, it made sense to use .length. The second clause checks if there's a response from the OMDb API.

### Email and password validation (users.js)

The validator NPM package<sup>25</sup> was used for checking for valid email addresses and strong passwords.

```
if (!validator.isEmail(email)) {
  return res.status(400).render("user/register", {
    invalidEmailError: "You must provide a valid email address",
  });
};

if (!validator.isStrongPassword(password)) {
  return res.status(400).render("user/register", {
    invalidPasswordError:
      "Password must be a minimum of 8 characters, with at least 1 uppercase letter, 1
number and 1 symbol"
  });
};
```

All possible errors across all endpoints have been configured with unique identifiers inside the res.render method to promote ease of use within the views engine and to display an appropriate message for the user.

## Security

Security was handled to the best of my current capabilities and understanding. It should be noted that my app has not been deployed to a URL. I chose not to do this because I would prefer to wait until I have a more in-depth understanding of security best practice. Nevertheless, the following measures were taken.

### HTTPS

Although it should be noted that the security provided by self-generated SSL certificates is limited to the development environment<sup>26</sup>, I did generate my own for my app. A further layer of security was added by adding the SSL files to .gitignore, and only referencing their file paths as environment variables. The code is configured to

---

<sup>24</sup> <https://knexjs.org/guide/query-builder.html#select>

<sup>25</sup> <https://www.npmjs.com/package/validator>

<sup>26</sup> [https://canvas.quantonline.edu.au/courses/1604/pages/8-dot-7-activity-2-https-in-express?module\\_item\\_id=92572](https://canvas.quantonline.edu.au/courses/1604/pages/8-dot-7-activity-2-https-in-express?module_item_id=92572)

start a HTTPS server or a HTTP server if the user decides not to generate the certificate.

### Environmental variables and .gitignore

Since I was pushing my project to a github repository, I took extra care in creating a .gitignore file and including everything I didn't want to appear on github, including my .env file. Environment variables have been created for port and database information, as well as the OMDb API key, the json web token secret and of course the SSL file key and certificate file paths.

### Middleware

Helmet<sup>27</sup> has been set up predominantly with its default variables, but with its 'img\_src' directive set to accept files from the URL that serves the movie posters. Rate limiting was implemented using express-rate-limit<sup>28</sup> to head off malicious attacks and server overloading<sup>29</sup>. Finally, validator was used to prevent Cross-Site Scripting<sup>30</sup> attacks at the registration endpoint, although future iterations of this project or other projects might include more rigorous validation for other endpoints to prevent script insertion.

## **Security – Assignment Questions:**

I now take the time to address the questions about security as required for this assignment.

### Why is HTTPS important?

HTTPS is important because it adds a layer of security on top of HTTP. In a normal HTTP request, a header is sent that contains information about a request (what the client wants to see or get from the website) or a response (what the server is configured to show or give the client). The issue here is that this can leave the server and/or the client open to malicious attacks if there is information in the header that an attacker can exploit. HTTPS uses TLS (Transport Layer Security) to encrypt the information contained in these headers, making it practically impossible for potential attackers to glean information from them<sup>31</sup>.

---

<sup>27</sup> <https://blog.logrocket.com/using-helmet-node-js-secure-application/>

<sup>28</sup> <https://www.npmjs.com/package/express-rate-limit>

<sup>29</sup> <https://blog.logrocket.com/rate-limiting-node-js/>

<sup>30</sup>

<https://www.digitalocean.com/community/tutorials/how-to-handle-form-inputs-efficiently-with-express-validator-in-express-js>

<sup>31</sup> [https://canvas.qutonline.edu.au/courses/1604/pages/8-dot-6-http-and-tls?module\\_item\\_id=92571](https://canvas.qutonline.edu.au/courses/1604/pages/8-dot-6-http-and-tls?module_item_id=92571)

### Why would you choose to use a certificate authority over a self-signed certificate?

While easy to generate, self-signed certificates are still vulnerable to attack. Attackers can replace your certificates with their own<sup>32</sup> – a tactic often used in man-in-the-middle attacks<sup>33</sup>. Another problem is that browsers will typically warn users before proceeding to websites with self-signed certificates.

A better solution is to attain a certificate signed by a certificate authority (CA). A CA is an organisation that will issue you with a certificate once they are sure your site is legitimate. Browsers have an index of trusted CAs and once you have a certificate bound to one of these CAs, users will be able to securely visit and interact with your site.

### If you were to deploy your API, which certificate authority would you use and why?

Choosing a CA would entail researching to make sure a particular authority is legitimate. This might involve visiting the websites of some authorities, reading other sources about them, and seeing if their certificates are already recognised on your machine. On a Mac, this last step can be done by looking at the “System Roots” panel in “Keychain Access”. Once I’m sure of the legitimacy of some CAs, the next questions I’d be asking myself would be which company offers the most favourable ratio of price to level of security and availability of customer service. Based on an article published on techradar.com, Comodo SSL seems to be a good option<sup>34</sup>.

---

<sup>32</sup> ibid

<sup>33</sup>

<https://web.synametrics.com/comparing-self-vs-ca-signed-ssl-certificate.htm#:~:text=Security%20%2D%20CA%20certificates%20are%20not,security%2C%20or%20credit%20card%20numbers.>

<sup>34</sup> <https://www.techradar.com/news/best-ssl-certificate-provider>

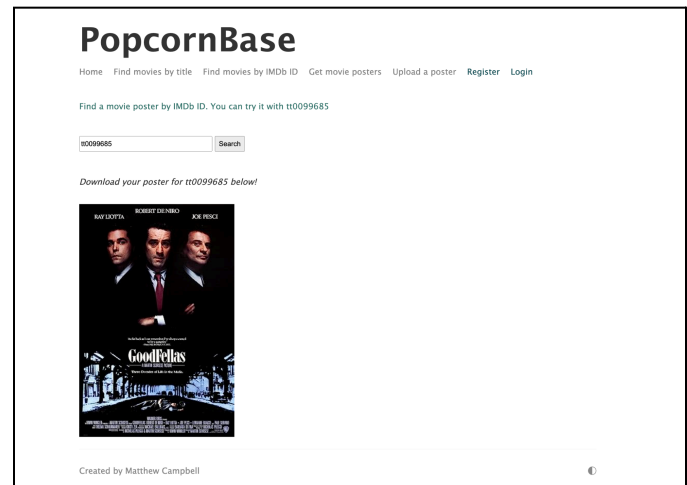
## References

- <http://54.79.30.138:3001/#/>
- <https://www.omdbapi.com/>
- [https://canvas.qutonline.edu.au/courses/1604/pages/5-dot-3-activity-1-installing-express?module\\_item\\_id=92543](https://canvas.qutonline.edu.au/courses/1604/pages/5-dot-3-activity-1-installing-express?module_item_id=92543)
- <https://www.udemy.com/course/the-complete-nodejs-developer-course-2/?couponCode=24T6MT62024>
- [https://www.w3schools.com/sql/sql\\_like.asp](https://www.w3schools.com/sql/sql_like.asp)
- <https://knexjs.org/guide/query-builder.html#where>
- <https://handlebarsjs.com/guide/builtin-helpers.html#unless>
- <https://www.sitepoint.com/a-beginners-guide-to-handlebars/>
- <https://stackoverflow.com/questions/12531743/handlebars-templating-and-dynamic-images>
- <https://www.npmjs.com/package/multer>
- <https://medium.com/@livajorge7/a-comprehensive-guide-to-uploading-and-saving-images-with-node-js-express-js-and-javascript-69137e227c4d>
- <https://code.likeagirl.io/can-sql-store-images-2266304dcddb>
- <https://dev.to/cesareferrari/creating-data-with-knex-in-a-rest-api-295e>
- <https://knexjs.org/guide/query-builder.html#update>
- [https://canvas.qutonline.edu.au/courses/1604/pages/7-dot-6-activity-2-registration-and-logi?module\\_item\\_id=92562](https://canvas.qutonline.edu.au/courses/1604/pages/7-dot-6-activity-2-registration-and-logi?module_item_id=92562)
- <https://www.digitalocean.com/community/tutorials/nodejs-jwt-expressjs>
- [https://help.sap.com/doc/abapdocu\\_753\\_index\\_hm/7.53/en-US/abenseparation\\_concerns\\_guidl.htm#:~:text=Separation%20of%20concerns%20is%20a,and%20arrangement%20in%20software%20layers.](https://help.sap.com/doc/abapdocu_753_index_hm/7.53/en-US/abenseparation_concerns_guidl.htm#:~:text=Separation%20of%20concerns%20is%20a,and%20arrangement%20in%20software%20layers.)
- <https://betterstack.com/community/guides/logging/how-to-install-setup-and-use-winston-and-morgan-to-log-node-js-applications/>
- <https://www.npmjs.com/package/helmet>
- <https://blog.logrocket.com/using-helmet-node-js-secure-application/>
- <https://lebcit.github.io/posts/csp-nonce-with-nodejs-and-ejs/>
- <https://johnkavanagh.co.uk/articles/check-if-a-string-contains-only-whitespace-with-javascript/>
- <https://knexjs.org/guide/query-builder.html#select>
- <https://www.npmjs.com/package/validator>
- [https://canvas.qutonline.edu.au/courses/1604/pages/8-dot-7-activity-2-https-in-express?module\\_item\\_id=92572](https://canvas.qutonline.edu.au/courses/1604/pages/8-dot-7-activity-2-https-in-express?module_item_id=92572)
- <https://blog.logrocket.com/using-helmet-node-js-secure-application/>
- <https://www.npmjs.com/package/express-rate-limit>
- <https://blog.logrocket.com/rate-limiting-node-js/>
- <https://www.digitalocean.com/community/tutorials/how-to-handle-form-inputs-efficiently-with-express-validator-in-express-js>
- [https://canvas.qutonline.edu.au/courses/1604/pages/8-dot-6-https-and-tls?module\\_item\\_id=92571](https://canvas.qutonline.edu.au/courses/1604/pages/8-dot-6-https-and-tls?module_item_id=92571)
- <https://web.synametrics.com/comparing-self-vs-ca-signed-ssl-certificate.htm#:~:text=Security%20%2D%20CA%20certificates%20are%20not,security%2C%20or%20credit%20card%20numbers.>
- <https://www.techradar.com/news/best-ssl-certificate-provider>

## User Guide

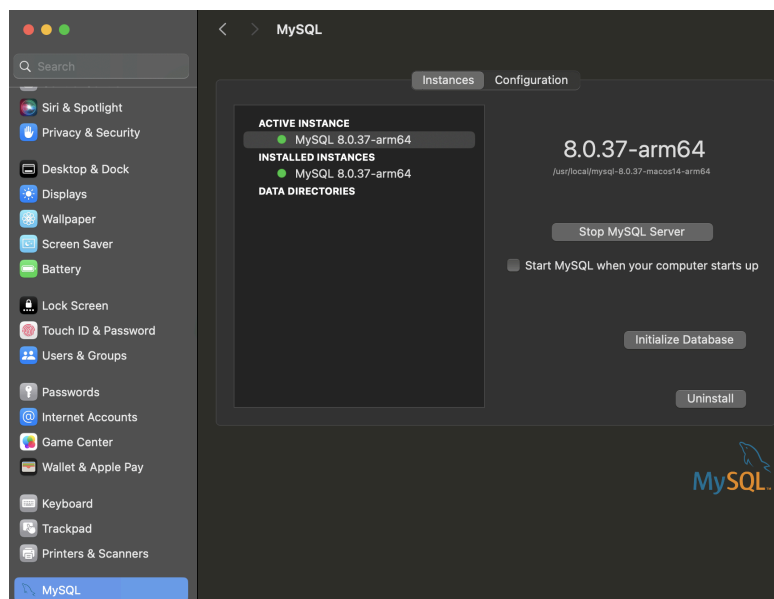
The following guide will take you through how to run the PopcornBase application. The guide assumes that you have downloaded the source code for the application, the `movies_small.sql` database file and the following applications:

- [Node and npm](#)
- [VS Code](#)
- [Insomnia](#)
- [MySQL](#)

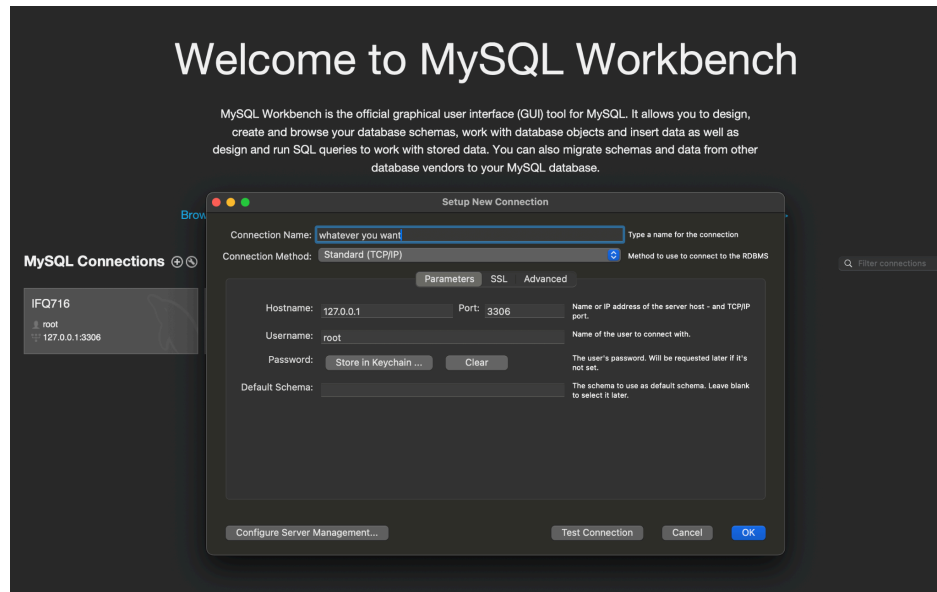


### Step 1: Fire up the MySQL Server and set up a new connection

Make sure you have a MySQL server instance running. On a Mac, this can be found in Settings once MySQL Server is installed. Click on 'Start MySQL Server' and enter your password.



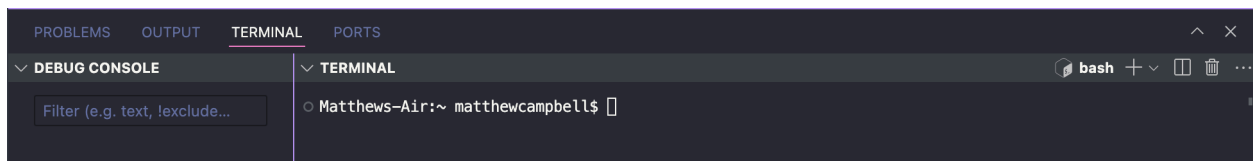
Once you've done that, open MySQL Workbench and create a new connection by clicking on the plus sign that appears next to "MySQL Connections". You should see something like this:



Give the connection whatever name you like. You can leave all other fields as their defaults. Then, hit 'OK'.

## Step 2: Install project dependencies

Open VS Code, then from your navigation bar at the top go to **File > Open Folder** and open the `movies-api` directory. Once it's open, start a new terminal again from the navigation bar: **Terminal > New Terminal**. You should have something like this:



In your terminal, run the following line and wait until the dependencies have finished installing:

```
npm install
```

### Step 3: Set up environment variables

There are a few steps here and we'll go through them one-by-one.

First, create a .env file by running this prompt in the terminal:

```
touch .env
```

Add the following fields in your newly created .env file:

```
PORT=3000

DB_HOST=[your ip address]
DB_NAME=movies
DB_USER=[your username]
DB_PASSWORD=[your password]

OMDB_KEY=[your OMDb key]

JWT_SECRET=[some random letters and digits]

SSL_KEY_PATH=[the path to your SSL key]
SSL_CERT_PATH=[the path to your SSL key]
```

The **DB (database)** information can be found in the main panel if you start up MySQL workbench. The username and password will be the ones you set up when installing MySQL or configuring the connection.

The **OMDb key** can be generated by going to the [OMDb API key generator page](#) and entering your email address where promoted. Copy and paste the key into the .env file.

For simplicity's sake, you can create the **JWT secret** simply by mashing keys on your keyboard. Make it a long string of numbers and letters.

For the SSL key and certificate, you'll need to do the following.

Once again in the terminal, run the following line:

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout selfsigned.key  
-out selfsigned.crt
```

You'll be prompted to enter some information, but just hit enter for each field until the two files appear in the project root directory.

Now, in your terminal, type in `pwd`. Copy and paste the path that comes up with the names of the two files into the `.env` file. It should look something like this:

```
SSL_KEY_PATH=path/to/your/sslkey/selfsigned.key  
SSL_CERT_PATH=path/to/your/sslcrt/selfsigned.crt
```

You can also choose not to create the SSL key and certificate if you want to, but it does ensure the app utilises HTTPS which is generally more secure than HTTP.

Leave out the square brackets (as used in the example above) for all fields.

#### **Step 4: Start the server and visit the link**

Now you're ready to test the app. Run the following in the terminal:

```
npm start
```

Depending on whether you chose to create the SSL key and certificates or not, you'll see either

```
Server running on https://localhost:3000
```

or

```
Server running on http://localhost:3000
```

The difference being one is https and the other is http. Copy and paste the link into your browser. If you're following the https link, you may see a warning message. This just means your browser doesn't trust self-created SSL certificates. Just go into your advanced settings and enable the browser to proceed to the link. For testing purposes

this is fine.

### Step 5: Have a look around!

You're now able to toy around with the app. Bear in mind the database is small, so some of your favourite movies might not be there. Feel free to test the app to its limits and see what happens!

# PopcornBase

[Home](#) [Find movies by title](#) [Find movies by IMDb ID](#) [Get movie posters](#) [Upload a poster](#) [Register](#) [Login](#)

Enter an imdb ID for information about a movie. Try this one if you like: tt0101540

Movie not found. Please try a different IMDb ID.